

ODI Demo Build — Trainer's Guide (3 sessions)

For you, the facilitator. You narrate and paste the prompts into chat; he copies them into Claude Code and clicks Allow.

How to run these sessions

You facilitate; he stays hands-on so he learns by doing. **You** handle every login, credential, and any red error. **He** pastes each prompt into Claude Code and clicks **Allow** when it asks. Read the **Say** line to frame each step, paste the blue prompt into chat for him to copy, and keep the grey notes to yourself. Pause whenever he's curious — letting him ask Claude *"explain that like I'm new to this"* is the best moment in the whole demo.

Lock these before session 1 (fill the blanks in every prompt below)

- **[VERTICAL]** — the industry story (e.g. Banking, Insurance)
- **[ORG]** — a made-up demo company name (generic; must not collide with a real entity)
- **[PERSONA]** — who the demo is for (e.g. a risk analyst)
- **[source]** — the app to pull from with OAuth (e.g. Salesforce, HubSpot, Google)

Your pre-session setup (every session, ~3 min)

- Fivetran API key in the shell: `export FIVETRAN_API_KEY=key:secret` (re-do each session — it doesn't survive a new terminal)
- AWS: `aws configure` then `verify aws sts get-caller-identity` (both are currently NOT set on this machine)
- Open the Fivetran dashboard and the AWS Glue/Athena console in browser tabs so he sees things appear live
- **If a login won't cooperate:** don't burn the session — have Claude tour a finished demo in `~/Documents/GitHub/*-ODI-Demo` and provision next time

If he asks what something means — your one-line answers

Connector	An automatic pipe that pulls data out of an app for you.
OAuth	The "Sign in with Google/Salesforce" button — he logs in, no passwords shared.
Sync	The connector copying the data over for the first time.
Data lake / MDLS	Cloud storage where all the data lands, in an open format anyone can read.
Iceberg	The modern open table format the data is stored as in the lake.
AWS	Amazon's cloud — S3 stores files, Glue lists tables, Athena queries, IAM is permissions.
Terraform	A tool that builds the cloud pieces automatically from a script Claude writes.
dbt	A tool that cleans and reshapes raw data into tidy tables for analysis.
Connect Card	A Fivetran browser link he opens to log into the source app with OAuth.

Session 1 (today) — set up the cloud and pull in the data

Your goal: AWS lake provisioned, source connected, first sync started. The Terraform step is the slow/risky one — protect time for it.

Step 1 — kick off the build

Say: "We tell Claude the whole goal and point it at a finished demo to copy the conventions from — then it lays out the plan before doing anything."

PASTE → he enters this in Claude Code:

Help me build an ODI demo from scratch — no special tooling, just you and me. End goal: an AWS Managed Data Lake destination in Fivetran, a source connector, synced data, a dbt bronze/silver/gold pipeline, and a small React web app. Use one of the finished demos in `~/Documents/GitHub/*-ODI-Demo` as the reference pattern for structure and conventions. Vertical: `[VERTICAL]`. Org name: `[ORG]`. Buyer persona: `[PERSONA]`. My AWS and Fivetran logins are configured. Lay out the plan, then walk me through it step by step in plain language.

You watch for: No skill needed — Claude reads a sibling demo in `~/Documents/GitHub/*-ODI-Demo` to mirror its structure (bronze/silver/gold, deep links, frontend), prints a plan, then starts scaffolding. Teaching moment: it's writing infrastructure-as-code itself.

Step 2 — build the cloud storage (the risky one)

Say: “Now it builds the storage on AWS automatically — this is the part that normally takes a team a day.”

PASTE → he enters this in Claude Code:

Scaffold the Terraform for the S3 bucket, the Glue databases, and the Fivetran IAM role, then walk me through applying it. Tell me when you need anything from me.

You watch for: The MDLS handshake: you start the destination in the Fivetran UI to get `aws_account_id` + `external_id`, paste them into `tfvars`, then it runs `terraform apply` (S3 + Glue + IAM). Most likely failure point: AWS perms or a stale `external_id`. Budget the most time here.

Step 3 — create the data lake destination

Say: “We point Fivetran at the lake we just built.”

PASTE → he enters this in Claude Code:

Create the Managed Data Lake destination via the REST API using the Terraform outputs, and show me it connected successfully.

You watch for: POST `/v1/destinations`; expect code `Created`. A permission error here almost always means the `external_id` in `tfvars` doesn't match the one from the destination setup UI.

Step 4 — connect the source app (OAuth)

Say: “Now we connect the source app. You'll sign in the normal way in your browser — no passwords go into Claude.”

PASTE → he enters this in Claude Code:

Create the [source] connector via the REST API, then give me a Fivetran Connect Card link so I can sign in with OAuth.

You watch for: Claude creates the connector and mints a Connect Card token; he opens the URL, does OAuth consent, picks tables, Saves. Teaching moment: OAuth shares access, not passwords. If your key lacks `connect-card` scope, finish auth in the Fivetran UI instead.

Step 5 — start the data flowing

Say: “Last step for today — kick off the first sync and watch the data land.”

PASTE → he enters this in Claude Code:

Run the connector setup test (expect `CONNECTED`), then trigger the first sync and poll until it's `SYNCED`. Tell me which tables landed.

You watch for: Keep the source selection small so this finishes fast. While it syncs, this is the spot for the Hybrid deployment talking point (agent runs in the customer's network; only metadata leaves).

SESSION DONE — the cloud lake is live, the app is connected, and data is flowing in. If Terraform ran long, stopping after the connector is created and the sync is triggered is a fine session 1.

Session 2 — shape the raw data into clean tables

Your prep: re-export the Fivetran key, re-verify `aws sts get-caller-identity`, confirm session 1's sync finished.

Step 1 — pick up where we left off

Say: “Let's make sure last session's data actually arrived before we build on it.”

PASTE → he enters this in Claude Code:

Resume the ODI demo we started. Confirm the Fivetran sync from last session finished and list the tables that landed in the Glue bronze database.

You watch for: Re-oriens Claude to the demo dir and proves the bronze tables exist. If the sync wasn't finished last time, this is where it completes.

Step 2 — build the data pipeline

Say: “Now we turn those raw tables into clean, business-ready ones — bronze, silver, gold.”

PASTE → he enters this in Claude Code:

Build the dbt project the same way the reference demo does — bronze sources, silver staging models, and one gold Iceberg fact/dimension. Keep it customer-ready: use the [ORG] name in anything user-facing, never raw table names. Explain each layer in plain language.

You watch for: Content standards: [ORG] everywhere, every connector keeps its Fivetran deep link, no raw table names exposed. Mirror the reference demo's dbt layout. Confirm the dbt-athena env vars (LAKE_BUCKET, ATHENA_WORKGROUP) are set.

Step 3 — run it and check the result

Say: “Run the pipeline and let's look at the finished data.”

PASTE → he enters this in Claude Code:

Run dbt build --select bronze silver gold and confirm the gold Iceberg tables were created in Glue. If it fails, explain the error simply and fix it, then show me a sample of the gold data via Athena.

You watch for: Common build failures: wrong awsdatacatalog database ref in profiles.yml, missing Athena workgroup, or an IAM gap. Claude diagnoses from the error message.

SESSION DONE — raw data has been turned into clean, query-ready gold Iceberg tables. Strong stopping point.

Session 3 — build the web app and tour the demo

Your prep: same quick restart — re-export the key, re-verify the AWS login.

Step 1 — generate the web app

Say: “Now we put a face on it — a small web app that presents the whole pipeline.”

PASTE → he enters this in Claude Code:

Generate the React frontend shell — a Landing page with the [ORG] hero and a data-flow diagram (Fivetran to S3/Iceberg to dbt to Athena), and a Pipeline page listing the connector with its Fivetran deep link. Match the layout of the reference demo's frontend.

You watch for: Produces the frontend/ shell. Connector entries must carry the Fivetran deep link per our content standard.

Step 2 — make it look sharp

Say: "Let's give it a real, distinctive look instead of a generic template."

PASTE → he enters this in Claude Code:

Give the web app a distinctive, professional design for the [VERTICAL] industry and [PERSONA] audience — strong typography, a cohesive color system, analytical charts instead of dense tables, and none of the generic AI-template look. Match the polish of the reference demo.

You watch for: Direct design pass, no skill required. Strong wow moment for a novice — the before/after is dramatic.

Step 3 — final quality check

Say: "Quick polish pass so nothing looks unfinished in front of an audience."

PASTE → he enters this in Claude Code:

Do a content QA pass: [ORG] in every title, H1, and navbar; no raw table names exposed anywhere; every connector shows its Fivetran deep link.

You watch for: Catches the content-standard misses that read as sloppy in a customer demo.

Step 4 — walk the demo

Say: "And here's the finished product — let's walk it end to end."

PASTE → he enters this in Claude Code:

Start the frontend and give me a guided tour from start to finish — the landing page, the data flow, and the data in the lake.

You watch for: Opens the app and narrates it. This is the artifact he'd actually show a customer; let him drive the click-through.

SESSION DONE — a demo-ready ODI app: live connector, cloud data lake, clean dbt pipeline, and a polished web app.

If a session stalls (trainer triage)

Protect the CORE path over polish. Priority order if time runs short: data in the lake (S1) > clean gold tables (S2) > web app (S3). Every green box above is a clean place to stop and resume next time. If AWS fights you, fall back to touring a finished demo and provision next session — never spend more than ~15 min debugging credentials live in front of him.